

Universidad EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Pregrado en Ingeniería de Sistemas

Organización de Computadores

Parcial 1 – Documento de Estrategia

Profesor

Edison Valencia Díaz

Integrantes

Mateo Montoya Ospina

Miguel Colorado Castaño

Sebastián Andrés Ibarra Prada

17 de septiembre de 2026

Contenido del Entregable

Diseño e Implementación de una ALU de 32 Bits	2
Diagrama de Circuito	4
Diseño de una ALU de 3 Entradas (X, Y, Z)	4
Conjunto de operaciones seleccionado	5
Análisis de arquitectura	8
Diagrama de Circuito	10

Diseño e Implementación de una ALU de 32 Bits

Para la creación de la ALU de 32 bits fue necesario plantear bien la manera en la cual se iba a adaptar la entrada de dos inputs de 32 bits a un software que únicamente acepta inputs de 16 bits. Por lo cual, se planteó la división de las entradas "x" e "y" en dos partes que abarcaran el inicio y el final de cada una de estas, contando con 16 bits, que es el máximo que acepta el programa. El inicio de ambas entradas es, respectivamente, "x0a15[16]" y "y0a15[16]"; y el final de ambas entradas es, respectivamente, "x16a31[16]" y "y16a31[16]". Además, se incorporaron los selectores para la ALU 32, como lo serían: zx (zero x), nx (negate x), zy (zero y), ny (negate y), f (function select) y no (negate output).

En lo que respecta a las salidas, se entendió que era necesario dividir la salida también en dos outputs debido a la limitación de 16 bits, por lo que se posee para la parte inicial "out1[16]" y para la parte final "out2[16]". Además, se cuenta con las banderas propias de la ALU, como lo son: zr (zero flag) y ng (negative flag).

Ahora, hablando del preprocesamiento de las variables (lo que se hace antes de la selección de función), se procede tanto con x como con y mediante el uso de dos

Mux16 para cada una, para decidir si se recibe el input —tanto superior como inferior— de x o un valor falso (false), y de igual manera el input superior e inferior de y o un false. Los inputs de x e y son decididos respectivamente por los selectores z_x y z_y . A estos valores que obtenemos de las elecciones los vamos a llamar: xz_0 (parte inicial x) y xz_1 (parte final x); yz_0 (parte inicial y) y yz_1 (parte final y).

Después de lo anterior, se procede a negar todos los outputs del proceso previo. Luego, se decide nuevamente con otros dos Mux16, tanto para x como para y, si se escoge la opción original de x (xz_0 y xz_1) o la versión negada, y si se escoge la versión original de y (yz_0 y yz_1) o la versión negada. Estas decisiones para x e y son tomadas respectivamente por n_x y n_y . También, los outputs serán llamados: xf_0 (inicial x), xf_1 (final x), yf_0 (inicial y) y yf_1 (final y).

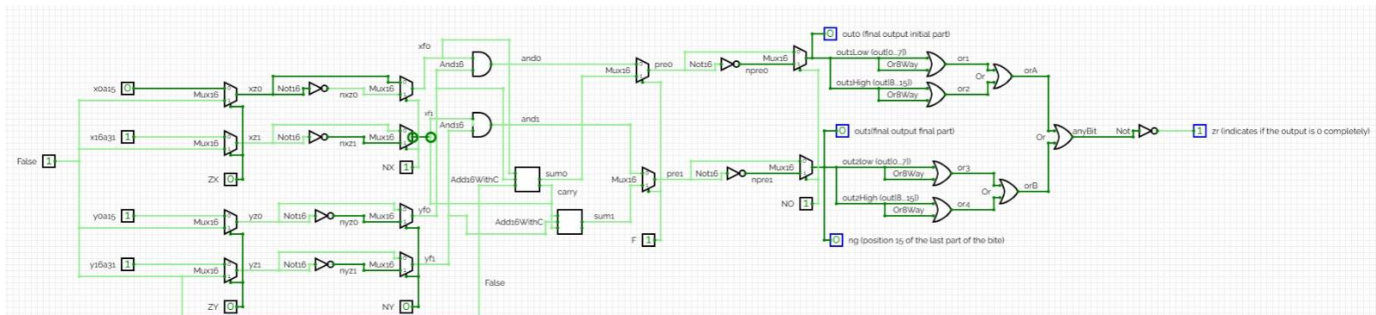
Tras haber finalizado el preprocesamiento de las variables, se toma la decisión de si escoger una operación tipo And o una operación tipo Add. Realizar la operación de And es sumamente sencillo, debido a que únicamente realizamos dos operaciones tipo And16. La primera se usará para realizar un And entre las partes iniciales de x e y, y el segundo And se usa para las partes finales de x e y. Mientras tanto, la parte del Add incluye una complejidad más amplia, debido a que se necesita realizar un proceso de carry (acarreo) entre las partes iniciales y las finales, además de considerar el caso en donde la suma supere la cantidad de bits esperada; por lo que se utiliza un Add16 con carry para poder solucionar este asunto, realizando la suma de la parte inferior y mandando el carry a la suma de las partes finales, para de este modo realizar todo el proceso de Add.

Después, en la ALU se utilizan dos Mux16 con el selector f para decidir si se escoge la operación Add o la operación And, siendo el primer Mux para la parte superior y el segundo para la parte inferior.

Después de haber escogido entre el Add y el And, se procede a la etapa de negar los outputs de salida; por último, negamos estas salidas por medio de un Not16 y, mediante dos Mux16, escogemos entre obtener la versión sin negar o la versión negada para así obtener el resultado final de la ALU, generando a su vez separaciones declaradas de 16 bits que luego se utilizarán para las banderas.

Ya al final, para determinar las banderas: para la ng simplemente revisamos el bit 15 de la parte final del output; y la zr se verifica utilizando cuatro Or8Way, los cuales realizan la operación Or con las partes iniciales y finales respectivas, juntando luego sus resultados para que al final, después de haber realizado múltiples Or, se pueda determinar si todo es 0 o no. Luego se niega este resultado, ya que si todo es 0 obtendremos un 0, por lo que la bandera se debe activar usando un 1; en caso contrario, si existe, aunque sea un solo 1, se negará el resultado, desactivando así

Diagrama de Circuito



Diseño de una ALU de 3 Entradas (X, Y, Z)

Conjunto de operaciones seleccionado

La ALU propuesta ejecuta 8 operaciones. Como existen ocho posibilidades diferentes, se requieren tres bits de control, debido a que $2^3 = 8$. La entrada op[3] se utiliza para seleccionar una de las ocho salidas calculadas por el circuito. El conjunto de operación seleccionadas es el siguiente:

Op[3]	Operación	Expresión
000	Suma triple	$X + Y + Z$
001	Resta combinada	$(X + Y) - Z$
010	And - Or	$(X \text{ AND } Y) \text{ OR } Z$
011	Or - And	$(X \text{ OR } Y) \text{ AND } Z$
100	And triple	$X \text{ AND } Y \text{ AND } Z$
101	Or triple	$X \text{ OR } Y \text{ OR } Z$
110	Xor triple	$X \text{ XOR } Y \text{ XOR } Z$
111	Mayoría bit a bit	$(X \text{ AND } Y) \text{ OR } (X \text{ AND } Z) \text{ OR } (Y \text{ AND } Z)$

Antes de escribir el HDL, primero se definió cómo se iba a construir el circuito teniendo en cuenta las restricciones de Nand2Tetris y las diferentes opciones que había para resolverlo. El primer problema fue combinar tres señales, ya que las compuertas y chips disponibles, como And16, Or16, Add16 y Xor, trabajan solamente con dos entradas, por lo que las operaciones con tres entradas tienen que hacerse en dos pasos. Por ejemplo, $X+Y+Z$ se calcula como $(X+Y)+Z$, mientras que $(X \text{ AND } Y) \text{ OR } Z$ se hace primero calculando $X \text{ AND } Y$ y luego haciendo la OR con Z .

Al revisar las 8 operaciones, se encontró que varias empiezan haciendo lo mismo. Las operaciones 000 y 001 comienzan con $X+Y$; las 010, 100 y 111 con $X \text{ AND } Y$; las 011 y 101 con $X \text{ OR } Y$; y la 110 con $X \text{ XOR } Y$. En vez de repetir estos cálculos para cada operación, se decidió hacerlos una sola vez al inicio del circuito y guardar sus resultados como addXY, andXY, orXY y xorXY. Después, estos resultados se utilizan en las diferentes ramas. Esto evita tener varias compuertas haciendo el mismo cálculo y hace que el circuito sea más sencillo.

Cabe aclarar que el hack de Nand2Tetris no tiene resta directa, por lo que para la operación 001 construimos $-Z$ negando bit a bit (Not16) y sumando 1 (Inc16) — complemento a 2 - con la entrada constante que se expone en el diagrama más abajo y luego se suma normal.

Para escoger cuál de las 8 operaciones debía salir, se consideraron dos opciones. Una era utilizar una estructura parecida a la ALU de Nand2Tetris, usando los bits de control zx, nx, zy, ny, f y no para modificar las entradas antes de llegar al resultado. La otra era calcular las 8 operaciones por separado y después seleccionar una

mediante un multiplexor. Se descartó la primera opción porque no todas las operaciones se pueden obtener simplemente modificando, anulando o negando X, Y y Z antes de hacer una suma o un AND, pues este método funciona para las funciones de la ALU estándar, pero no se adapta directamente a operaciones como la 111, que trabaja bit a bit con las tres entradas, por lo que se escogió la segunda opción que es calcular las 8 operaciones en paralelo y seleccionar el resultado al final. Además, como son exactamente 8 operaciones, se puede utilizar directamente el Mux8Way16 que ya proporciona Nand2Tetris, sin tener que construir un chip adicional.

Después se decidió cómo calcular las banderas zr y ng. Estas se obtienen después del Mux8Way16, usando solamente el resultado que fue seleccionado. No tendría sentido calcularlas en cada una de las 8 ramas porque los otros resultados finalmente se descartan. De esta forma solo se necesita un circuito para cada bandera.

Para ng es bastante sencillo, ya que, en un número de 16 bits representado en complemento a dos, el bit 15 indica si el número es negativo. Por eso, se puede tomar directamente ese bit de la salida del multiplexor. Para zr hay que revisar los 16 bits para saber si todos son cero y como Or8Way solo recibe 8 entradas, se divide la salida en dos grupos de 8 bits, se aplica un Or8Way a cada grupo, se combinan los dos resultados con una Or y se niega el resultado. Así, zr será 1 solamente cuando los 16 bits de la salida sean cero.

Por último, fue necesario construir el chip auxiliar Xor16, porque Nand2Tetris no lo proporciona directamente, solo se cuenta con el Xor de una entrada de un bit, por

lo que se replica este mismo Xor en las 16 posiciones para obtener el Xor16. De esta manera se puede utilizar la operación XOR de 16 bits dentro del circuito principal.

Análisis de arquitectura

Si bien el diseño presentado demuestra que una ALU de tres operandos es viable a nivel de compuertas, su extensión a un procesador real plantea dificultades importantes. En primer lugar, una instrucción con tres operandos fuente y un destino exige codificar cuatro direcciones de registro; en un banco de 32 registros, esto implica veinte bits dedicados exclusivamente al direccionamiento, dejando un margen reducido para el código de operación dentro de una instrucción de longitud fija. Como las instrucciones tienen un tamaño fijo, esto deja muy poco espacio para el resto de la información que la instrucción también debe llevar, como el tipo de operación a realizar. Esto obliga a elegir entre instrucciones más largas, o a permitir muy pocas operaciones de tres operandos en el procesador.

La segunda dificultad es el consumo de energía. Cada entrada adicional que recibe la ALU obliga a agregar una vía de lectura adicional en el banco de registros, y cada vía nueva requiere más cableado y más circuitos conectados a todos los registros, lo cual consume más energía. En el diseño presentado, esto se refleja en que las ocho operaciones definidas se calculan en paralelo todo el tiempo, aunque solo una de ellas se use en cada momento; las siete restantes consumen energía sin necesidad. Reutilizar resultados intermedios, como se hizo en este diseño, ayuda a reducir este consumo, aunque el valor exacto de energía dependería de cómo se fabrique finalmente el circuito.

En tercer lugar, la unidad de control debe generar un mayor número de señales simultáneas para gestionar tres operandos fuente, y en un procesador segmentado esto puede aumentar las demandas sobre las rutas de datos y los mecanismos de

reenvío; aun así el efecto sobre los riesgos de datos depende de la organización específica del pipeline y no debe considerarse automático en todo diseño.

Una ALU de tres operandos puede ser útil cuando una operación específica se utiliza con frecuencia y su ejecución directa evita varias instrucciones adicionales. Un ejemplo es la operación de multiplicación y suma fusionada, $a \cdot b + c$, utilizada en determinados procesadores para acelerar cálculos numéricos. Por lo tanto, la conveniencia de una ALU de tres entradas depende de los beneficios obtenidos frente al costo de hardware, energía y complejidad de control.

Desde el punto de vista arquitectónico, una ALU de tres operandos implica analizar el formato de las instrucciones, la lectura de registros, el consumo de energía y la complejidad de la unidad de control. Aunque puede representar un costo adicional frente a una ALU convencional de dos entradas, su utilización puede justificarse cuando permite acelerar operaciones importantes para una aplicación o arquitectura específica.

Diagrama de Circuito

Enlace al repositorio: <https://github.com/The-G-Man-Half-Life/ALUProject.git>

